Neuromorphic Intermediate Representation (NIR)

A common language for neuromorphic systems

Jens E. Pedersen · jegpe@dtu.dk

DTU Electro

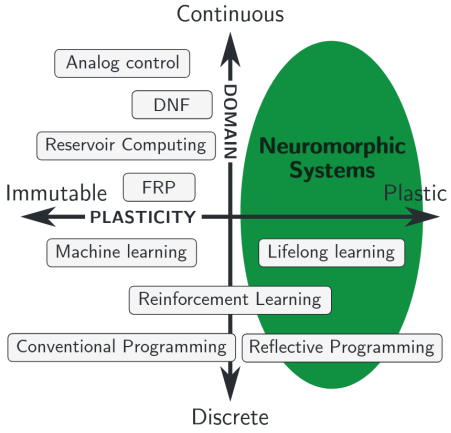
ISCAS 2026 Tutorial — SNNs to Silicon

The problem: a fragmented ecosystem

Training an SNN today means **choosing a stack**:

- Framework: Norse, snnTorch, Lava-DL, Sinabs, Spyx, ...
- Hardware: Loihi, BrainScaleS, SpiNNaker, Speck, Xylo, FPGA, ...
- Each combination requires **rewriting** the model

This is the same problem conventional ML solved with **ONNX** — but neuromorphic models have **dynamics** that ONNX cannot capture.

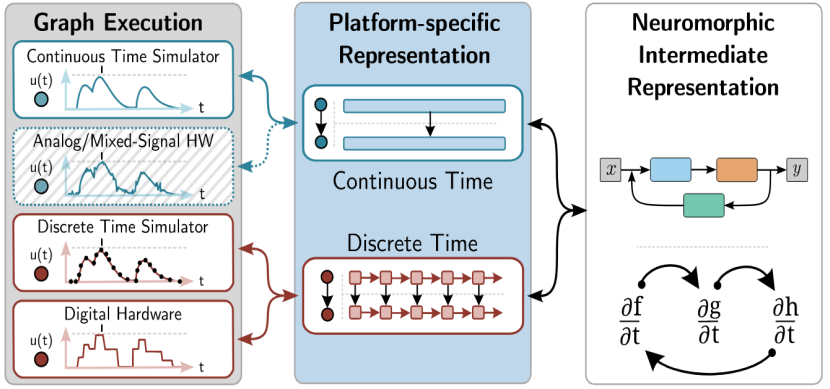


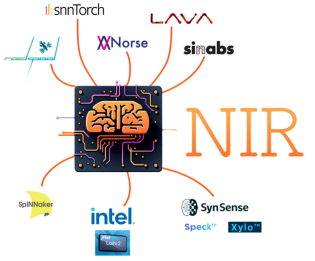
Pedersen et al., Nature Reviews Electrical Engineering, 2025

NIR: one model, many platforms

NIR captures SNN models as **graphs of continuous-time dynamical systems**

- Primitives are **parameterized ODEs** — no discretization baked in
- The graph is **substrate-agnostic**
- Each backend instantiates the dynamics:
  - Software → any ODE solver or Euler step
  - Digital HW → fixed-point discretization
  - Analog HW → physics implements the ODE
  - **FPGA** → streaming dataflow (NIR2FPGA)





What's in a NIR graph?

17 primitives covering the neuromorphic toolbox:

Category	Primitives
Neurons	LIF, LI, IF, CubaLIF, CubaLI
Synapses	Linear, Affine, Conv1d, Conv2d
Pooling	SumPool2d, AvgPool2d
Utility	Scale, Delay, Flatten, Threshold
I/O	Input, Output

The **LIF neuron** in NIR:

$$\tau \frac{dv}{dt} = -(v - v_{\text{leak}}) + R \cdot I$$

$$\text{if } v \geq \theta: \ \text{spike}, v \leftarrow v_{\text{reset}}$$

Parameters: τ, R, v_{leak}, v_{reset}, θ

→ Fully defines the dynamics, independent of timestep or solver

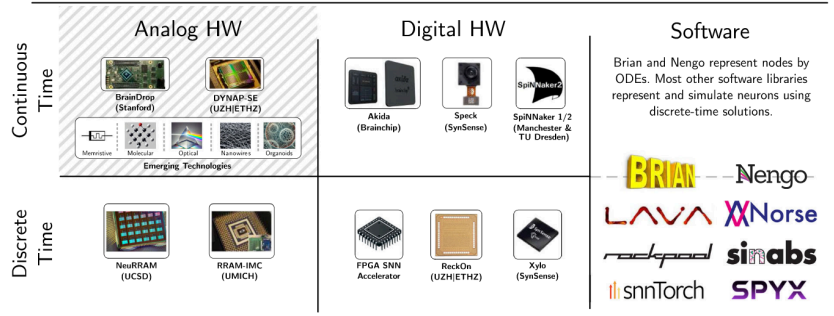
Supported platforms

10 software frameworks

- Norse (PyTorch)
- snnTorch (PyTorch)
- Sinabs (PyTorch / SynSense)
- Lava-DL (Intel)
- Rockpool (SynSense)
- Nengo
- Spyx (JAX)
- jaxsnn (BrainScaleS)
- hxtorch (BrainScaleS)
- Spiking Jelly

5+ hardware targets

- Intel Loihi
- SynSense Speck & Xylo
- BrainScaleS-2
- SpiNNaker2
- FPGA via NIR2FPGA ← this tutorial!



NIR in practice: export and import

Export from any supported framework:

```python
import nir
import norse

# Train your model in Norse
net = norse.torch.SequentialState( ... )
#  ...  training loop  ...

# Export to NIR
nir_graph = norse.torch.to_nir(net)
nir.write("my_snn.nir", nir_graph)
```

Import into any target:

```python
import nir

# Load the NIR graph
graph = nir.read("my_snn.nir")

# Inspect the model
for name, node in graph.nodes.items():
    print(f"{name}: {type(node).__name__}")
    # e.g., "lif_0: LIF"
    # e.g., "linear_0: Affine"

# Deploy to hardware, FPGA, or simulator
```

File format is **HDF5** — portable, self-describing, and language-agnostic

Building a NIR graph from scratch

```
import nir
import numpy as np

graph = nir.NIRGraph(
    nodes={
        "input":  nir.Input({"input": np.array([784])}),
        "linear": nir.Affine(weight=np.random.randn(128, 784),
                             bias=np.zeros(128)),
        "lif":    nir.LIF(tau=np.full(128, 20.0),
                          r=np.ones(128),
                          v_leak=np.zeros(128),
                          v_threshold=np.ones(128),
                          v_reset=np.zeros(128)),
        "out":    nir.Affine(weight=np.random.randn(10, 128),
                             bias=np.zeros(10)),
        "output": nir.Output({"output": np.array([10])}),
    },
    edges=[("input", "linear"), ("linear", "lif"),
           ("lif", "out"), ("out", "output")]
)
nir.write("mnist_snn.nir", graph)  # Ready for NIR2FPGA!
```

synfire.dev: the neuromorphic model registry

An open platform for sharing and deploying SNN models

- **Model registry**: publish and discover SNNs
- **Built on NIR**: models are stored as graphs
- **Hardware-aware deployment**: deploy to neuromorphic chips and FPGAs
- **CLI + SDK + Web**: multiple developer interfaces

Think of it as **HuggingFace for neuromorphic AI** — train once, deploy everywhere

Workflow:

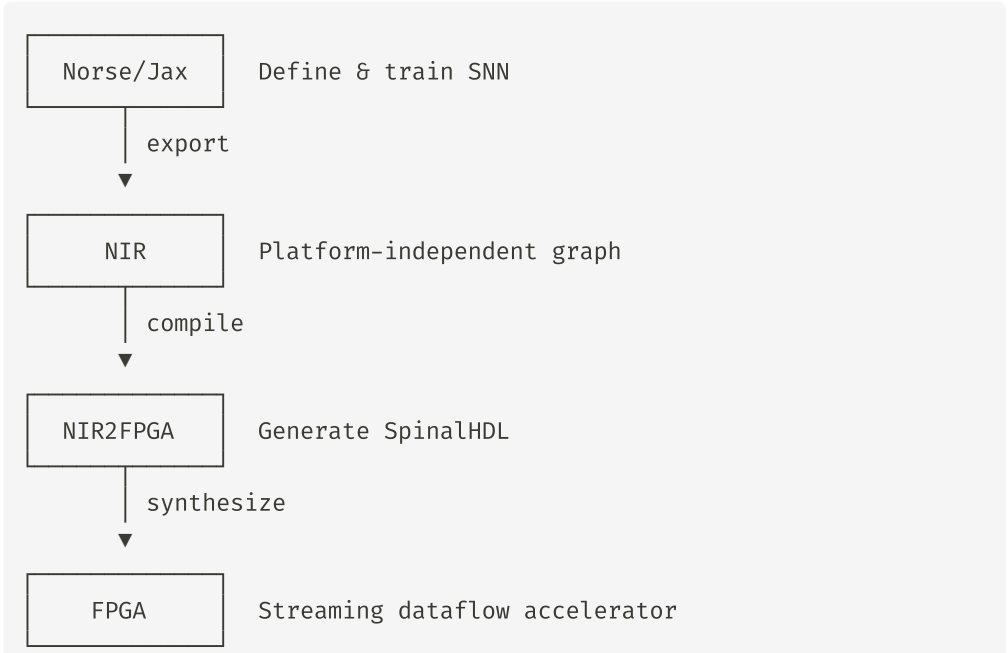
1. Train SNN in your preferred framework
2. Export to NIR
3. Upload to synfire.dev
4. Others discover & deploy your model
5. Target any supported hardware

The NIR2FPGA pipeline

This tutorial's end-to-end flow:

1. **Define** an SNN (JAX — next session)
2. **Train** with surrogate gradients
3. **Export** to NIR
4. **Compile** NIR → SpinalHDL RTL
5. **Simulate** or synthesize for FPGA

NIR closes the gap between **software simulation** and **hardware realization** — same model, verified at every step



Summary & resources

NIR is the common language for neuromorphic systems:

- 17 ODE-based primitives
- 10 frameworks, 5 (now 6!) hardware targets
- HDF5 file format, open source
- **synfire.dev** for model sharing
- **NIR2FPGA** for FPGA deployment

References & links

- neuroir.org — NIR documentation
- synfire.dev — model registry
- github.com/neuromorphs/NIR
- Paper: Neuromorphic Intermediate Representation, Nat. Commun, 2025
- Paper: Neuromorphic Programming, ICONS, 2024
- snnbook.net — free SNN textbook

Next up: Defining, training & exporting SNNs →